



Universidad Nacional Autónoma de México
Facultad de Ingeniería

Laboratorio de Computación Gráfica

Estadio de fútbol
Recreación 3D interactiva en OpenGL

Manual de usuario y manual técnico

Integrantes: Nava Santiago Erick
Tenorio Martínez Jesús Alejandro

Profesor: Ing. Edén Espinoza Urzúa

Asignatura: Laboratorio de Computación Gráfica

Semestre: 2027-1

Fecha de entrega: 9 de noviembre de 2026

Ciudad Universitaria, Ciudad de México

Índice

Resumen 1

I Manual de usuario 1

1. Introducción 1

2. Requisitos e instalación 1
2.1. Requisitos . . . 1
2.2. Compilación y ejecución . . . 2

3. Controles 2

4. Secuencia del partido 3

5. Recorrido recomendado para la demostración 4

6. Solución de problemas 5

II Manual técnico 9

7. Objetivos 9
7.1. Objetivo general . . . 9
7.2. Objetivos específicos . . . 9

8. Alcance 9

9. Limitantes 10

10. Metodología de software aplicada 11

11. Arquitectura del software 12
11.1. Archivos y responsabilidades . . . 12
11.2. Bucle principal . . . 12

12. Diagrama de flujo del software 13

13. Diagrama de Gantt 13

14. Documentación del código 14
14.1. Estados del partido . . . 14
14.2. Tiro penal . . . 15
14.3. Llegada de autobuses . . . 15
14.4. Funciones de dibujo . . . 16
14.5. Sistema de cámaras . . . 16

14.6. Iluminación y cielo 16
14.7. Texturizado procedural 17
14.8. Modelos importados y recursos 17
14.9. Uso del código base del profesor 17
14.10Pruebas 18

15.Modelos y escena **19**

16.Materiales y texturas **21**

17.Conclusiones **22**
17.1. Erick Nava Santiago 22
17.2. Jesús Alejandro Tenorio Martínez 23
17.3. Conclusión general 23

Referencias **23**

Resumen

Este documento reúne el manual de usuario y el manual técnico de *graficaLaboratorio*, una recreación tridimensional de un estadio de fútbol nocturno desarrollada en C++ con OpenGL 3.3. La escena se compone de geometría generada por código (cancha, pista atlética, gradas, fachada, porterías, jugadores, árbitro, público, árboles, torres de iluminación, drones, ave y trofeo) y de modelos OBJ importados con Assimp (cámaras de transmisión y autobuses de los dos equipos). Sobre esa base se programó una secuencia de partido con máquina de estados (llegada, saque, goles, penales y final), cinco animaciones por código y un sistema de cámaras interactivo. El manual de usuario explica la instalación, los controles y un recorrido de demostración; el manual técnico documenta objetivos, alcance, limitantes, metodología, arquitectura, diagrama de flujo, cronograma, materiales, pruebas y conclusiones.

Parte I

Manual de usuario

1. Introducción

El programa muestra un estadio abierto de noche, con dos equipos identificados como Pumas y Atlante, público en las dos gradas laterales y una secuencia de partido que puede reproducirse de forma automática o recorrerse escena por escena. El usuario puede observar la escena desde una cámara libre, desde dos drones o desde siete vistas de transmisión, y puede activar la llegada de los autobuses, el saque inicial, los goles, la tanda de penales, la ceremonia final y los fuegos artificiales.

La aplicación está pensada como demostración académica: no es un videojuego, no tiene audio ni condiciones de victoria jugables. Su propósito es exhibir el pipeline gráfico completo (modelado, texturizado, iluminación, animación, interacción y renderizado en tiempo real).

2. Requisitos e instalación

2.1. Requisitos

- Sistema operativo Linux con controladores OpenGL 3.3 Core Profile o superior.
- Compilador de C++ con soporte para C++11 (`g++`).
- Bibliotecas GLFW 3, GLEW, GLM y Assimp, además de `pkg-config`.
- Una GPU capaz de sostener la resolución del monitor a una tasa aceptable. La escena usa ocho luces puntuales, cuatro reflectores y sombreado por fragmento.

2.2. Compilación y ejecución

El proyecto incluye `Configinicial/build.sh`, que compila la biblioteca SOIL2 incluida y enlaza el ejecutable sin depender de los archivos de Visual Studio. En Ubuntu o Debian, las dependencias se instalan con:

```
sudo apt install build-essential pkg-config libglfw3-dev libglew-dev libglm-dev libassimp
    -dev
cd Configinicial
./build.sh
./graficaLaboratorio
```

El ejecutable debe lanzarse desde la carpeta `Configinicial/`, porque todas las rutas de shaders, texturas y modelos son relativas (`Shader/`, `Models/`). Si se ejecuta desde otra carpeta, el programa arranca pero reporta recursos faltantes y sustituye algunas texturas por colores planos.

3. Controles

La ventana abre a la resolución del monitor y el título muestra el marcador, el tiempo, la escena actual, el estado de pausa y los penales. La proyección se ajusta al tamaño del framebuffer cuando la ventana cambia. La tabla 1 resume todas las teclas implementadas en `main.cpp`.

Cuadro 1: Controles del programa.

Tecla	Acción	Contexto
`Esc`	Cierra la ventana.	Siempre
`0`	Activa la cámara libre y reinicia la orientación del ratón.	Siempre
Ratón	Gira la cámara libre (yaw y pitch).	Cámara 0
`W/S`	Avanza o retrocede.	Cámara 0
`A/D` o ←/→	Se desplaza a la izquierda o a la derecha.	Cámara 0
`1`, `2`	Selecciona el dron 1 o el dron 2.	Siempre
`W/S/A/D`	Desplaza el dron en el plano horizontal.	Drones 1–2
`Q/E`	Baja o sube el dron.	Drones 1–2
`Shift`	Duplica la velocidad del dron.	Drones 1–2
`3`–`8`	Selecciona una cámara fija de transmisión; el cabezal sigue al balón.	Siempre
`9`	Vista aérea cenital; en la final apunta al trofeo.	Siempre
`Z`	Inicia la llegada de los autobuses.	Desde el estado inicial
`L`	Reinicia y reproduce la secuencia completa automática.	Siempre
`Espacio`	Inicia la secuencia automática si está detenida; en otro caso pausa o reanuda.	Siempre
`X`	Salta al saque inicial.	Siempre

Tecla	Acción	Contexto
C	Salta al gol de Pumas.	Siempre
V	Salta al gol de Atlante.	Siempre
N	Salta a la tanda de penales.	Siempre
P	Ejecuta un tiro penal.	Escena de penales
M	Salta a la final con fuegos artificiales.	Siempre
F	Lanza fuegos artificiales desde cualquier escena.	Siempre
R	Reinicia el partido y vuelve al estado inicial.	Siempre
H	Enciende o apaga los faros de los autobuses.	Siempre
↑/↓	Aumenta o reduce la intensidad de las cuatro torres.	Siempre
[/]	Gira 90° los modelos de cámara de transmisión.	Siempre

4. Secuencia del partido

El programa implementa seis escenas además del estado inicial. Cada cambio se hace con una transición de fundido; la tabla 2 describe qué ocurre en cada una.

Escena	Tecla	Contenido
Inicial	R	Estadio nocturno, público en las gradas, trofeo en el pedestal y drones en vuelo.
Llegada	Z	Los dos autobuses entran por los accesos, giran, se detienen y los jugadores bajan y forman.
Saque	X	Balón al centro; el jugador 10 de Atlante ejecuta el saque y el balón avanza por el campo.
Gol de Pumas	C	Tiro de esquina, dos remates de cabeza y gol; el público de Pumas festeja y el rival reacciona.
Gol de Atlante	V	Jugada de Atlante, remate y gol; se invierte el festejo.
Penales	N	Tanda con portero animado, marcador de penales y reacciones según el resultado.
Final	M	El capitán levanta el trofeo, el público celebra y se lanzan fuegos artificiales.

Cuadro 2: Escenas y teclas de acceso.

Con L, o con Espacio desde el estado inicial, la secuencia avanza sola: llegada, saque, gol de Pumas, gol de Atlante, penales y final. En los penales automáticos el programa dispara cada intento y decide al ganador. Si se entra con N, la secuencia no es automática y cada tiro debe ordenarse con P; esto permite explicar la animación paso a paso.



Figura 1: Vista general del estadio desde la cámara libre: cancha, pista atlética, gradas, fachada y torres de iluminación.



Figura 2: Fachada perimetral y accesos vistos desde el exterior. La llegada de los autobuses usa estos accesos.

5. Recorrido recomendado para la demostración

El siguiente recorrido cubre los requisitos de la entrega en menos de cinco minutos:

1. Iniciar el programa y observar la vista aérea con `9`. Comentar la planta del estadio, la pista y las cuatro torres.
2. Pulsar `0` y recorrer la fachada con `W/A/S/D` y el ratón (figura 2).
3. Volver a una vista fija con `5` y pulsar `Z` para la llegada. Encender los faros con `H` (figura 3).
4. Pulsar `L` para reproducir la secuencia completa. Dejar que se vean el saque, los dos goles y el festejo del público (figuras 4 y 5).
5. En los penales, si se desea control manual, pulsar `N` y luego `P` para cada tiro (figura 6).
6. Pulsar `M` para la final y los fuegos (figuras 7 y 8).
7. Cerrar con `Esc`.

6. Solución de problemas

- **La ventana no abre o se cierra de inmediato.** Verificar que el equipo tenga controladores OpenGL 3.3 y que el ejecutable se lance desde `Configinicial/`.
- **Aparecen mensajes `Texture not loaded` o `Missing team bus`.** Faltan archivos de `Models/`. Revisar que la carpeta completa se haya copiado y que las rutas conserven mayúsculas y minúsculas.
- **Algunos objetos se ven de color plano.** Es el respaldo previsto cuando una textura no se encuentra; el programa sigue funcionando con `CreateSolidColorTexture`.
- **Rendimiento bajo.** La escena dibuja cientos de cubos y carga modelos OBJ pesados. Cerrar otras aplicaciones y usar una resolución de ventana menor ayuda; la geometría no se simplifica en tiempo de ejecución.
- **Los penales no avanzan solos.** Si se entró con `N`, hay que pulsar `P` en cada intento. La secuencia automática solo se activa con `L` o `Espacio`.



Figura 3: Llegada de un autobús por el acceso lateral, con faros encendidos.



Figura 4: Público en la grada de Pumas: playeras, rostros diferenciados y butacas de madera.



Figura 5: Árbitro con playera amarilla texturizada durante el saque inicial.



Figura 6: Tiro penal: balón, portero y portería con red texturizada.



Figura 7: Trofeo sobre su pedestal, con mapas de oro, mármol y madera.



Figura 8: Fuegos artificiales calculados por código durante la ceremonia final.

Parte II

Manual técnico

7. Objetivos

7.1. Objetivo general

Aplicar los conocimientos del curso de Computación Gráfica para construir, documentar y exponer una recreación tridimensional interactiva de un estadio de fútbol, usando OpenGL 3.3, geometría generada por código, modelos importados, texturizado, iluminación, animación por estados e interacción en tiempo real.

7.2. Objetivos específicos

- Recrear una fachada y un espacio con al menos siete tipos de objetos distintos, con geometría y texturizado adecuados.
- Conservar y extender la base docente de las prácticas (contexto OpenGL, cámara, shaders, carga de modelos, texturizado, iluminación y máquinas de estados).
- Programar al menos cinco animaciones: tres sencillas y dos complejas.
- Implementar una secuencia de partido coherente con el tema, con transiciones y estados.
- Documentar el uso del programa, la arquitectura, el flujo, el cronograma, el alcance, las limitantes, las pruebas y las conclusiones.

8. Alcance

El sistema cubre una cancha de fútbol con pista atlética, gradas laterales, fachada perimetral con dos accesos, cuatro torres de iluminación, árboles, porterías, marcador, dos equipos de once jugadores, árbitro, público, cámaras de transmisión, dos drones, un ave, un trofeo y dos autobuses importados. La interacción se limita a recorrer la escena, cambiar de cámara, controlar la secuencia y ajustar la iluminación; no hay edición de la escena, audio, colisiones ni modo multijugador.

La tabla 3 enumera los tipos de objeto con texturizado de imagen. Los objetos repetidos (jugadores, butacas, árboles, torres) cuentan como un solo tipo, y los detalles de ambientación no se usan para completar el mínimo de siete.

Tipo de objeto	Texturas principales
Cancha	`Grass005_Color`
Pista atlética	`running_track_diff_1k`
Gradas y butacas	`gravel_concrete_03`, `old_wooden_floor_02`
Fachada y accesos	`concrete_diff_1k`, `worn_shutter_diff_1k`
Porterías	`metal_plate_02`, `fabric_leather_01`
Balón	`leather_white`, `fabric_leather_01`
Jugadores	Playeras PNG, atlas de rostro, shorts y piel planos
Árbitro	`referee_yellow_jersey`, atlas de rostro
Público	Playeras PNG y cuatro atlas de rostro
Árboles	`pine_bark`, `LeafSet024_Dark/Light`
Torres de iluminación	`worn_shutter_diff_1k`
Cámaras de transmisión	Modelos OBJ con logo Liga MX
Autobuses	Carrocerías y ruedas OBJ con sus mapas
Drones	`blue_metal_plate`, `metal_plate_04`, `rubber_tiles`
Ave	`fabric_leather_02`, `rough_wood`
Trofeo	`trophy_gold`, `marble_01`, `dark_wood`

Cuadro 3: Tipos de objeto y texturas asociadas.

9. Limitantes

- **Texturizado procedural.** La geometría creada con cubos y esferas no tiene UV modelados a mano. El sombreador calcula coordenadas a partir de la posición o la normal, de modo que en algunas caras el mapa se estira o se repite. Los uniformes, la piel, el calzado y varios accesorios siguen usando color plano.
- **Cámaras fijas.** Las cámaras 3–8 apuntan siempre al balón y no permiten encuadres libres. La cámara 9 es una vista cenital, no una vista de fachada; para observar la fachada se usa la cámara libre. La redistribución de cámaras propuesta en la rama `camaras` no está integrada.
- **Sin sombras proyectadas.** La iluminación es local (Phong/Blinn-Phong); no hay mapas de sombra ni oclusión ambiental.
- **Sin audio ni físicas.** No hay sonido, colisiones, rebotes reales ni simulación de multitudes.
- **Dependencia de rutas relativas.** El ejecutable debe correr desde `Configinicial/`.
- **Plataforma.** La compilación reproducible documentada es Linux mediante `build.sh`; el proyecto de Visual Studio existe, pero no se validó en esta entrega.
- **Repetición de geometría.** El público y los jugadores se dibujan con cubos y esferas reutilizados; no son mallas esculpidas.

10. Metodología de software aplicada

Se siguió un desarrollo incremental con control de versiones Git y ramas por aportación:

1. **Análisis de la consigna y diseño.** Lectura de los requisitos, definición de la planta del estadio y del conjunto de escenas.
2. **Base gráfica.** Contexto OpenGL 3.3, ventana GLFW, entrada, cámara, shaders de iluminación y carga de modelos con Assimp.
3. **Modelado por código.** Cancha, pista, gradas, fachada, porterías, jugadores, árbitro, público, árboles, torres, drones, ave y trofeo.
4. **Texturizado e iluminación.** Mapas de color CC0, atlas de rostros, playeras generadas por script y luces puntuales y reflectores.
5. **Animación y estados.** Llegada de autobuses, saque, goles, penales, final, festejos y fuegos artificiales.
6. **Verificación.** Compilación con `build.sh` y quince scripts de comprobación en `Configinicial/tests/`.
7. **Documentación.** Este manual, el archivo de pendientes y los comentarios del código.

El historial de Git se usó como registro real de avances: primer commit el 28 de octubre de 2026, base avanzada el 5 de noviembre, texturas y escena el 7 y 8 de noviembre, y animaciones finales el 9 de noviembre. Los commits se agruparon por ramas de trabajo (`erick-jisus`, `rama-chui`, `camaras`, `limpieza-repo`) y se integraron en `main`.

11. Arquitectura del software

11.1. Archivos y responsabilidades

Archivo o carpeta	Responsabilidad
`main.cpp`	Inicialización, recursos, bucle, entrada, escenas, animaciones y dibujo.
`Camera.h`	Cámara con vectores front/up/right y ángulos de Euler (base docente).
`Shader.h`	Compilación, enlace y activación de programas GLSL (base docente).
`Model.h`, `Mesh.h`	Carga y dibujo de mallas OBJ con Assimp (base docente).
`Shader/lighting.*`	Iluminación Phong/Blinn-Phong, ocho luces puntuales y cuatro reflectores.
`Shader/sky.*`	Cielo nocturno procedural con estrellas y luna.
`Shader/lamp.*`	Dibujo de las lámparas de las torres.
`Models/`	Modelos OBJ, mapas de color, playeras y atlas de rostros.
`SOIL2/`	Carga de imágenes a texturas OpenGL.
`tests/`	Quince comprobaciones de código, texturas y geometría.
`build.sh`	Compilación reproducible en Linux.

Cuadro 4: Organización del proyecto.

11.2. Bucle principal

El programa usa tres programas GLSL: `lightingShader` para la escena, `lampShader` para las lámparas y `skyShader` para el fondo. En cada cuadro se leen los eventos, se actualiza el tiempo y, si la secuencia no está pausada, se ejecutan el movimiento de cámara o dron, la lógica de la escena, la animación del penal y la de los autobuses. Después se configuran cámara y luces, se dibuja el cielo sin escritura de profundidad, se dibuja la escena con profundidad y se intercambian los búferes.

```
glfwPollEvents();
if (!sequencePaused) {
    animationClock += deltaTime;
    DoMovement();
    updateMatch(deltaTime);
    if (matchScene == SCENE_PENALTIES && fadePhase != FADE_OUT) Animation();
    if (matchScene == SCENE_ARRIVAL && fadePhase != FADE_OUT) updateBusAnimation();
    updateFade(deltaTime);
}
view = getBroadcastView();
projection = glm::perspective(glm::radians(fov), aspect, 0.1f, 300.0f);
// cielo, escena, HUD y swap buffers
```

12. Diagrama de flujo del software

La figura 9 muestra el flujo por cuadro. La decisión de pausa omite las actualizaciones pero mantiene el dibujo, de modo que la ventana sigue respondiendo. La decisión de escena activa controla la máquina de estados del partido; el estado inicial solo dibuja la escena y el trofeo.

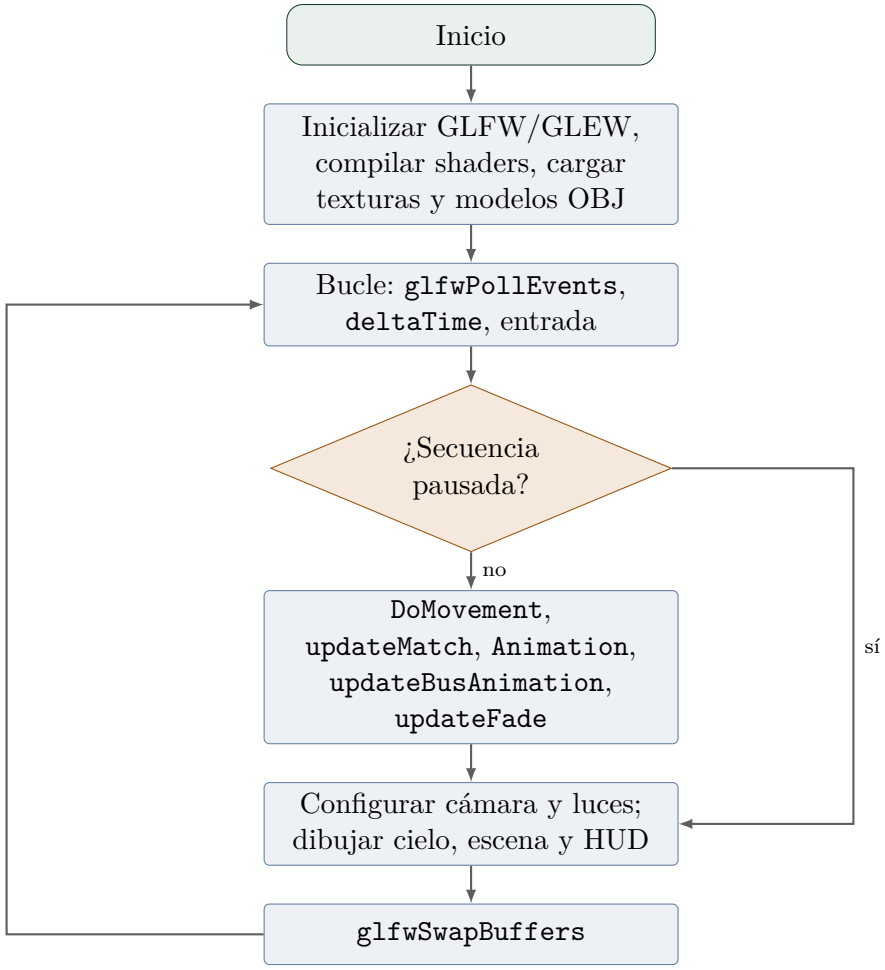


Figura 9: Diagrama de flujo del programa. El ciclo se repite hasta que se cierra la ventana.

13. Diagrama de Gantt

El cronograma de la figura 10 usa las fechas reales del historial de Git. La escala comienza el 28 de octubre de 2026 y termina el 9 de noviembre, fecha de entrega.

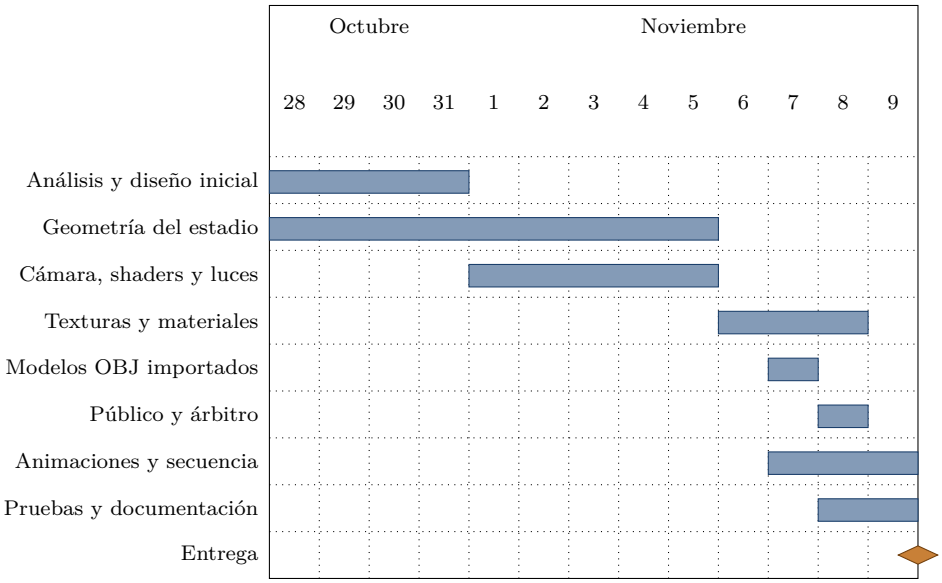


Figura 10: Cronograma del proyecto con fechas del historial de Git.

Actividad	**Periodo**	**Responsable**
Análisis de la consigna y diseño inicial	28–31 oct	Erick Nava
Geometría del estadio y la cancha	28 oct–5 nov	Erick Nava
Cámara, shaders e iluminación	1–5 nov	Erick Nava
Texturas y materiales	6–8 nov	Jesús Tenorio
Modelos OBJ de cámaras y autobuses	7 nov	Erick Nava
Público, árbitro y reacciones	8 nov	Jesús Tenorio
Animaciones y secuencia de partido	7–9 nov	Ambos
Pruebas, documentación y entrega	8–9 nov	Ambos

Cuadro 5: Actividades, periodos y responsables.

14. Documentación del código

14.1. Estados del partido

La máquina de estados principal usa la enumeración `MatchScene`. La tabla 6 resume las transiciones automáticas y manuales.

Estado	Contenido	Salida
SCENE_IDLE	Estadio, trofeo y drones.	Z, L, Espacio o salto directo.
SCENE_ARRIVAL	Autobuses y formación.	Automática al completar la formación; o X.
SCENE_KICKOFF	Saque y desplazamiento del balón.	Automática a los 6.2 s; o C.
SCENE_PUMAS_GOAL	Jugada, remates y festejo.	Automática a los 8.3 s; o V.
SCENE_ATLANTE_GOAL	Jugada y festejo visitante.	Automática a los 6.5 s; o N.
SCENE_PENALTIES	Tanda, portero y marcador.	Automática al decidir ganador; o M.
SCENE_FINAL	Trofeo, celebración y fuegos.	R para reiniciar.

Cuadro 6: Estados de la secuencia de partido.

Cada cambio pasa por `requestMatchScene`: si la escena actual es el estado inicial se cambia de inmediato; en otro caso se activa `FADE_OUT`, se dibuja un velo negro y, al llegar a opacidad total, `beginMatchScene` reinicia el tiempo y entra con `FADE_IN`. La variable `autoSequence` decide si las escenas se encadenan solas o esperan una tecla.

14.2. Tiro penal

El penal usa la enumeración `PenaltyState`: `READY`, `PREPARING`, `KICKING`, `FLYING` y `FINISHED`. En `PREPARING` el tirador retrocede; en `KICKING` se aplica el ángulo de patada y el portero elige lado; en `FLYING` el balón interpola hacia un objetivo aleatorio dentro de la portería y el portero extiende el cuerpo; en `FINISHED` se registra gol o atajada, se actualizan los marcadores y se decide el ganador cuando la ventaja es matemáticamente irreversible. El límite de intentos por equipo es 64.

14.3. Llegada de autobuses

Cada autobús recorre una recta de 28 unidades y después un arco de 90° con radio 8 hasta quedar alineado con la cancha. La posición y el giro se calculan con `distanceTraveled`, `busSpeed` y `busTurnRadius`; las ruedas giran según la velocidad y el volante. Al detenerse, los once jugadores de cada equipo bajan con un retardo de 0.62 s y caminan 3.7 s hasta su formación. Los faros se encienden con `H` y la animación se activa con `Z`, `L` o `Espacio`.

14.4. Funciones de dibujo

Función	Propósito
`drawField`	Césped, líneas y áreas de la cancha.
`drawRunningTrack`	Pista atlética perimetral con esquinas redondeadas.
`drawPerimeter`	Muros, fachada, piso exterior y accesos.
`drawGoal`	Postes y red de las dos porterías.
`drawBleachers`	Gradas, butacas y pasillos.
`drawPlayer`	Jugador articulado con playera, cabeza, shorts, piel y calzado.
`drawGoalkeeperSave`	Portero con estirada y guantes.
`drawCelebrationCaptain`	Capitán que levanta el trofeo.
`drawCrowd`	120 aficionados por grada con cuatro rostros y festejo.
`drawTrees`	Troncos y copas con mapas de corteza y hojas.
`drawDrone`	Dron con cuatro rotores giratorios y lente.
`drawBird`	Ave que orbita y aletea.
`drawTrophy`	Trofeo con base, copa y asas.
`drawFireworks`	Tres estallidos de partículas calculadas.
`drawMatchScoreboard`	Marcador 3D con dígitos de siete segmentos.
`drawBroadcastCameraModels`	Cámaras OBJ con cabezal que sigue al balón.
`drawFloodlightTowers`	Cuatro torres con paneles de iluminación.
`updateMatch`	Máquina de estados y trayectorias del balón.
`Animation`	Máquina de estados del penal.
`updateBusAnimation`	Ruta, ruedas y descenso de jugadores.

Cuadro 7: Funciones principales de dibujo y actualización.

14.5. Sistema de cámaras

La cámara libre usa `Camera` (base docente) con posición, yaw y pitch; se mueve con `W/A/S/D` y se orienta con el ratón. Los drones 1 y 2 se desplazan con las mismas teclas, `Q/E` y `Shift`; sus posiciones se limitan a $x \in [-64, 64]$, $y \in [7, 34]$ y $z \in [-47, 47]$. Las cámaras 3–8 son modelos OBJ colocados en posiciones fijas; su cabezal calcula yaw y pitch hacia `trackedBallPosition`, que se suaviza con una interpolación exponencial. La cámara 9 es cenital y en la final apunta al trofeo.

14.6. Iluminación y cielo

`Shader/lighting.frag` implementa el modelo de Phong/Blinn-Phong con luz ambiental, difusa y especular, ocho luces puntuales y cuatro reflectores. Las cuatro torres aportan los reflectores; las flechas arriba/abajo escalan su intensidad entre 0.15 y 2.5 sin afectar los faros de los autobuses. `Shader/sky.frag` reconstruye la dirección del rayo con la inversa de la proyección y de la rotación de la vista, genera un degradado nocturno, estrellas por *hashing* de celdas y una luna con sombra. El cielo se dibuja primero, sin prueba de profundidad, para que no tape la escena.

14.7. Texturizado procedural

La geometría por código no tiene coordenadas UV explícitas. El sombreador de vértices `Shader/lighting.vs` selecciona un modo con el uniforme `proceduralTextureMode` y calcula `TexCoords` a partir de la posición o la normal. La tabla 8 describe los modos usados.

Modo	Uso	Coordenadas
0	UV explícitos	Atributo `texCoords` (modelos importados y caras con UV).
1	Cancha	`(x,z)` escalado 12×8.
2	Muros	Proyección lateral y altura.
3	Fachada y paneles	`(x,y)` centrado.
4	Piso exterior	`(x,z)` escalado 20×16.
5	Gradas	Triplanar en coordenadas de mundo, escala 1/4.
6	Esferas	Longitud y latitud de la normal, cuatro repeticiones.
7	Cajas	Triplanar de posición local, escala 2.

Cuadro 8: Modos de texturizado procedural.

```
if (proceduralTextureMode == 1)
    TexCoords = (position.xz + vec2(0.5)) * vec2(12.0, 8.0);
else if (proceduralTextureMode == 5)
    TexCoords = (abs(normal.y) > 0.5 ? FragPos.xz :
                 vec2(abs(normal.x) > 0.5 ? FragPos.z : FragPos.x, FragPos.y)) / 4.0;
else if (proceduralTextureMode == 6)
    TexCoords = vec2(atan(normal.z, normal.x) * 0.1591549 + 0.5,
                     asin(clamp(normal.y, -1.0, 1.0)) * 0.3183098 + 0.5) * 4.0;
else
    TexCoords = texCoords;
```

14.8. Modelos importados y recursos

Se importan dos carrocerías de autobús, seis ruedas y dos piezas de cámara de transmisión (`cameraBase.obj` y `cameraHead.obj`). `VerifyModelTextures` revisa que las rutas declaradas en los MTL existan antes de cargar. El modelo más pesado de `Models/` mide aproximadamente 69 MB, por debajo del límite de 100 MB por modelo. Las texturas de color provienen de ambientCG y Poly Haven con licencia CC0; las playeras, los rostros y el logo se generan o adaptan en el proyecto.

14.9. Uso del código base del profesor

El proyecto parte de la base docente de las prácticas P1–P10 y la conserva como esqueleto:

- **P1:** inicialización de GLFW/GLEW, contexto OpenGL 3.3, doble búfer y callbacks.
- **P2–P3:** pipeline de transformaciones con GLM, `Shader.h` y VAO/VBO del cubo.
- **P4:** modelado jerárquico para las extremidades de jugadores y árbitro.

- **P5 y P8:** carga de modelos con Assimp (`Model.h`, `Mesh.h`).
- **P6:** texturizado con SOIL2, UV y atlas.
- **P7–P8:** cámara, iluminación Phong/Blinn-Phong y shaders `lighting`/`lamp`.
- **P9–P10:** interpolación temporal y máquinas de estados para las animaciones.

La base se extendió con el cielo procedural, los modos de textura, el público, el árbitro, las reacciones, los autobuses y la secuencia de partido. Los archivos de la base que el proyecto usa directamente son `Camera.h`, `Shader.h`, `Model.h`, `Mesh.h` y los shaders de iluminación y lámpara.

14.10. Pruebas

La carpeta `Configinicial/tests/` contiene quince scripts de verificación. No usan un marco de pruebas unitarias: leen el código o las imágenes y fallan con una aserción cuando falta un recurso, una ruta o una propiedad geométrica.

Script	**Qué comprueba**
`check_ball_and_goalkeeper_textures.py`	Texturas del balón y playera distinta del portero.
`check_ball_visibility.py`	El balón aparece en juego y no en llegada o festejos.
`check_camera_logo.py`	Ruta y UV del logo Liga MX en la cámara.
`check_crowd_celebration.py`	Solo festeja la grada que anota o gana.
`check_crowd.py`	Playeras y rostros distintos en ambas gradas.
`check_drone_and_bird_textures.py`	Mapas PBR de drones y ave.
`check_goal_textures.py`	Texturas de postes y red con mapeo triplanar.
`check_night_sky.py`	El cielo sigue a la vista sin tapar el estadio.
`check_player_head.py`	Atlas de rostro generado sin escribir en recursos.
`check_pumas_goal_contact.py`	Contacto del balón en córner, remates y tiro.
`check_referee.py`	Playera reproducible y un árbitro por escena.
`check_rival_goal_emotions.py`	Emociones del rival al recibir gol.
`check_running_track.py`	Pista alineada a los cuatro lados sin cubrir la cancha.
`check_stadium_materials.py`	Mapas de gradas y butacas; atenuación de torres.
`check_trophy_textures.py`	Oro, mármol y madera del trofeo.

Cuadro 9: Scripts de verificación.

15. Modelos y escena

La figura 11 reúne los modelos que no forman parte de la arquitectura del estadio: la cámara de transmisión importada, un dron, el ave, el trofeo, el árbitro y un autobús. Las figuras 12 a 14 muestran cada uno por separado.

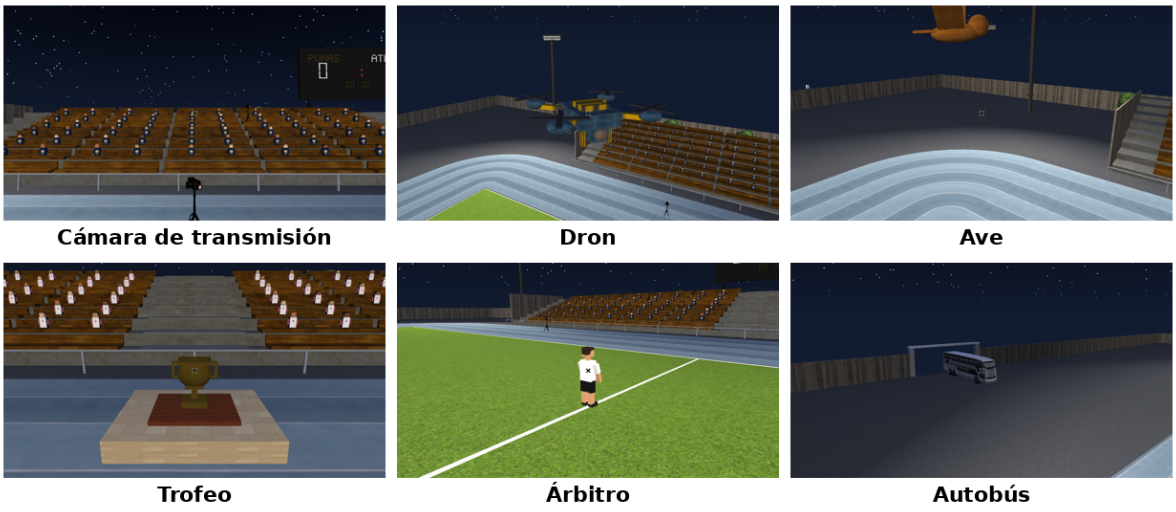


Figura 11: Lámina de modelos: cámara OBJ, dron, ave, trofeo, árbitro y autobús.



Figura 12: Cámara de transmisión importada con Assimp; el cabezal apunta al balón.

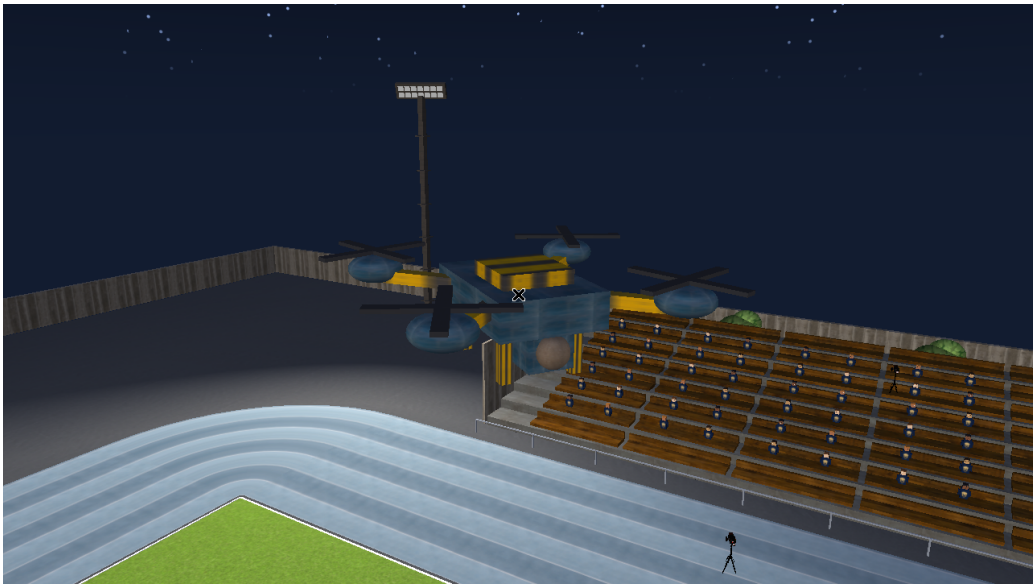


Figura 13: Dron de vigilancia con rotores animados y lente metálica.



Figura 14: Ave en órbita sobre el estadio; el aleteo se calcula con funciones seno.



Figura 15: Vista cenital de la cámara 9: planta del estadio, pista y distribución de las gradas.

16. Materiales y texturas

La figura 16 presenta una selección de los mapas de color usados en la escena. Los mapas de ambientCG y Poly Haven son CC0; las playeras de Pumas, Atlante, portero y árbitro, así como los atlas de rostro, se generan con los scripts de `Models/Textures/`. El logo de Liga MX se aplica al cabezal de la cámara de transmisión.

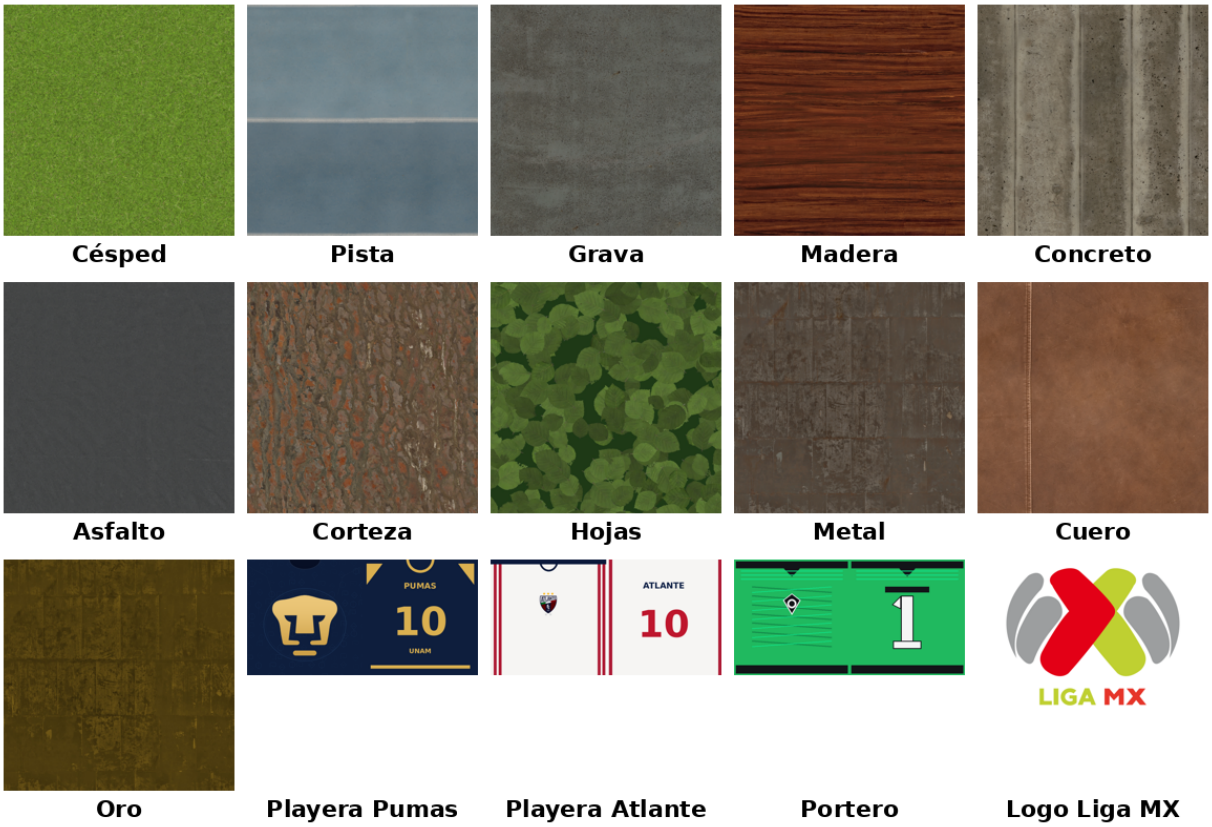


Figura 16: Lámina de texturas: césped, pista, grava, madera, concreto, asfalto, corteza, hojas, metal, cuero, oro, playeras y logo.

El sombreador recibe dos unidades de textura: el mapa difuso en la unidad 0 y un mapa especular compartido en la unidad 1. Los modelos importados usan sus propios materiales y UV; la geometría por código usa los modos procedurales de la tabla 8. Cuando una imagen no se encuentra, `LoadDiffuseTexture` devuelve 0 y el programa usa `CreateSolidColorTexture` para no dejar la cara sin material.

17. Conclusiones

17.1. Erick Nava Santiago

El proyecto permitió integrar en un solo programa los temas del curso: contexto OpenGL, cámara, transformaciones, carga de modelos con Assimp, texturizado, iluminación y máquinas de estados. La parte más demandante fue mantener coherentes la secuencia de partido, las trayectorias del balón y las cámaras que lo siguen, porque cualquier cambio de tiempo afecta varias escenas. La compilación reproducible con `build.sh` y los scripts de verificación redujeron el riesgo de romper recursos al integrar ramas. Queda como trabajo futuro el texturizado con UV modelados y una distribución de cámaras más cinematográfica.

17.2. Jesús Alejandro Tenorio Martínez

La aportación se centró en los materiales y en la vida de la escena: texturas de cancha, pista, gradas, árboles, público, árbitro y reacciones. El reto principal fue que la geometría por código no tiene UV, así que fue necesario proyectar las texturas desde la posición o la normal y aceptar sus limitaciones. Los atlas de rostro y las playeras generadas por script facilitaron variar a los personajes sin modelar cada cara. La experiencia confirma que el texturizado y la iluminación importan tanto como la geometría para que una escena se lea como un estadio y no como un conjunto de cubos.

17.3. Conclusión general

El sistema cumple los objetivos planteados: recrea un espacio reconocible con más de siete tipos de objetos texturizados, conserva y extiende la base docente, incluye cinco animaciones por código y ofrece una interacción suficiente para recorrer y demostrar la escena. Las limitantes documentadas —texturizado procedural, cámaras fijas y ausencia de sombras— no impiden la demostración, pero marcan el camino para una siguiente versión.

Referencias

[1] Espinoza Urzúa, E. *Proyecto Final. Laboratorio de Computación Gráfica*. Facultad de Ingeniería, UNAM, 2026. Documento de la asignatura.

[2] Khronos Group. *OpenGL 3.3 Core Profile Specification*. https://www.opengl.org/.

[3] GLFW. *Window and input library*. https://www.glfw.org/.

[4] Assimp. *Open Asset Import Library*. https://assimp.org/.

[5] G-Truc. *OpenGL Mathematics (GLM)*. https://glm.g-truc.net/.

[6] ambientCG. *Grass005, LeafSet024, Gravel Concrete 03*. Licencia CC0. https://ambientcg.com/.

[7] Poly Haven. *Pine Bark, Old Wooden Floor 02, Metal Plate, Marble, Trophy Gold*. Licencia CC0. https://polyhaven.com/.

[8] Documentación del código base de las prácticas P1–P10 del Laboratorio de Computación Gráfica, Facultad de Ingeniería, UNAM.